

Simulação de um Sistema Operacional Multitarefa de Tempo Compartilhado

Versão 0.5

Autor

Prof. Dr. Marco Aurélio Wehrmeister

Objetivos

1. Aprofundar o estudo do gerenciamento de tarefas através de implementação de escalonadores, a execução de múltiplas tarefas e do escalador, e a apresentação gráfica dessa execução do sistema ao longo do tempo.
2. Aprimorar o conhecimento sobre interação entre tarefas usando serviços de exclusão mútua comumente encontrados em sistemas operacionais multi-tarefa;
3. Compreender e resolver os problemas de escalonamento, p.ex., inanição, inversão de prioridades, impasse, etc.

Requisitos Gerais

Todos os requisitos abaixo devem ser obrigatoriamente contemplados em AMBOS os projetos:

1. O projeto deve implementar um **software que simula a execução** das tarefas em um *sistema operacional multitarefa preemptivo de tempo compartilhado*;
2. O computador possui **mais de um processador capaz de executar as tarefas** do sistema. A quantidade de processadores é configurada pelo usuário, sendo, no mínimo, dois e no máximo N processadores.
3. O software deve permitir a visualização gráfica da execução das tarefas dentro do sistema operacional ao longo do tempo para cada processador. **Todos os eventos importantes/relevantes** que acontecem ao longo da execução das tarefas devem **aparecer claramente na visualização gráfica** na forma de um elemento gráfico distintivo para cada tipo de evento. Uma legenda destes elementos gráficos deve ser mostrada na visualização. A visualização deve indicar, claramente, em qual processador cada tarefa está executando.

4. O software deve ser configurável e permitir que o usuário controle a simulação do sistema, bem como as características do sistema que está sendo simulado. Deve ser possível avançar e retroceder a simulação, sendo que o estado atual de cada tarefa pode ser modificado manualmente em qualquer passo/estágio da simulação.
5. A linguagem a ser utilizada para implementação do projeto é de livre escolha, no entanto, é necessário que o software criado possa ser executado sem a necessidade de instalar nenhuma biblioteca extra, ou qualquer outro software/runtime, ou equivalente.
6. Todo o código fonte deve ser comentado de forma que qualquer outra pessoa consiga compreender o que cada parte/função/componente de código faz e as razões/justificativas de todas as decisões/escolhas de implementação;

PROJETO A – Implementação Inicial do Simulador

Requisitos:

1. O projeto deve implementar um software que simula a execução de tarefas em um sistema operacional multitarefa preemptivo de tempo compartilhado.
 - 1.1. O sistema simulado deve **incluir um relógio global do sistema**, sendo que na passagem do tempo deve ser simulada na forma de *ticks* de relógio.
 - 1.2. O sistema possui **múltiplos processadores/CPU's/cores para executar as tarefas**. O escalonador deve **minimizar a ociosidade destes processadores**, ou seja, não devem existir tarefas na fila de prontos e processadores sem uso. Caso não seja possível atribuir uma tarefa para executar em um processador, o processador deve ser desligado. A informação sobre o período de tempo que um processador está desligado deve ser apresentada para o usuário e também na visualização gráfica da execução das tarefas.
 - 1.3. As informações de cada tarefa (antes, durante, depois da simulação) devem ser armazenadas durante a simulação em uma única estrutura de dados, p.ex., *Task Control Block* (TCB);
 - 1.4. A simulação da execução das tarefas deve seguir as características de tempo informadas para cada tarefa, p.ex., instante de ingresso, duração, instante da ocorrência de eventos, etc.
 - 1.5. **Dois modos de execução** da simulação devem ser implementados: (a) execução **passo-a-passo**; (b) execução **completa** sem intervenção humana.
 - 1.5.1. No caso execução passo-a-passo (opção a), deve ser possível examinar o estado atual e as informações correntes do sistema e também de cada tarefa individualmente. A ideia é que esse modo de execução forneça um *debugger* do sistema.
 - 1.5.2. No caso execução passo-a-passo (opção a), o simulador deve permitir que o usuário avance e retroceda a simulação conforme seu desejo. Dessa forma, o histórico do estado do sistema, a cada passo da execução da simulação, deve ser armazenado na memória e deve ser usado no processo de retroceder/avançar a simulação. **Retroceder/avançar a simulação tem impacto no relógio global do sistema**. Além disso, simulador deve permitir que o usuário modifique o estado atual de qualquer tarefa do sistema em qualquer passo/estágio da simulação.

1.5.3. No caso execução completa (opção b), o software mostra apenas o resultado final da simulação do sistema. Não é necessário mostrar os passos intermediários da simulação.

2. O software deve permitir a visualização gráfica da execução das tarefas dentro do sistema operacional ao longo do tempo.

2.1. Apresentar um gráfico de gantt que mostra as tarefas sendo executadas ao longo do tempo e o processador usado em cada período de execução. Quando uma tarefa está executando, tal informação deve ser indicada com uma cor (definida em um parâmetro de entrada) e cada tarefa deve ter uma cor diferente. Quando uma tarefa está na fila de prontas e ela não está sendo executada, tal informação deve ser indicada como a ausência da cor. Quando uma tarefa está suspensa, por qualquer motivo, tal informação deve ser indicada com a cor preta.

2.2. Quando uma tarefa nova chega no sistema isso deve ser claramente mostrado no gráfico de gantt. Da mesma forma, quando a tarefa termina a sua execução, isso deve ser claramente mostrado no gráfico de gantt. Para isso utilize ícones, figuras ou outro elemento gráfico que indique esses eventos.

2.3. A atualização do gráfico de gantt deve ser compatível com o modo de execução da simulação (conforme requisito 1.4). Na execução passo a passo, o gráfico de gantt atualizado deve ser mostrado na tela para o usuário a cada passo da simulação.

2.4. Ao final da simulação, o software deve gerar um arquivo do tipo imagem (p.ex. JPG, PNG, SVG, etc) contendo o gráfico de gantt correspondendo a situação final do sistema após a execução de todas as tarefas.

2.5. A visualização do gráfico de gantt deve ser similar as figuras da seção 6.4 do livro do Prof. Maziero. A ordem em que as tarefas aparecem no eixo Y deve ser decrescente em relação ao ID, ou seja, a tarefa com ID menor é a mais próximas do eixo X, a tarefa com o segundo ID menor aparece em cima da primeira, e assim por diante.

3. O software deve ser configurável e permitir que o usuário controle a simulação do sistema, bem como as características do sistema que está sendo simulado.

3.1. O software deve permitir a configuração/parametrização do sistema a ser simulado antes de execução da simulação, incluindo, mas não limitado a: conjunto de tarefas, características individuais de cada tarefa, algoritmo de escalonamento, eventos como solicitação/liberação de um mutex, operação de entrada/saída, envio/recebimento de dados, entre outros.

3.2. O software deve sugerir valores padrão para todo os parâmetros de configuração da simulação ou do software em si. Esses valores padrão devem ser sobrescritos por valores indicados pelo usuário;

3.3. A configuração/parametrização do sistema a ser simulado deve ser carregada a partir de um arquivo texto, formato texto simples (plain text), e deve seguir o seguinte formato:

```
algoritmo_escalamento;quantum;qtde_cpus  
id;cor;ingresso;duracao;prioridade;lista_eventos
```

onde:

`algoritmo_escalamento` indica o algoritmo de escalonamento a ser usado na simulação;

`quantum` indica o período máximo de tempo que uma tarefa pode executar;

`qtde_cpus` é a quantidade de processadores que o sistema possui para executar as tarefas;

`id` é o identificador único da tarefa pode executar;

`cor` é cor que identifica a execução da tarefa no formato RGB em

Hexadecimal, p.ex. “F0E0D0” indica Red=F0, Green=E0, Blue=D0

`ingresso` indica o instante de tempo que a tarefa foi criada;

`duracao` indica o tempo de execução da tarefa;

`prioridade` indica a prioridade da tarefa;

`lista_eventos` indica uma lista de eventos que ocorre durante a execução da tarefa.

3.3.1. A primeira linha do arquivo de configuração indica parâmetros gerais da simulação ou do sistema operacional. A partir da segunda linha, cada linha indica os parâmetros de uma tarefa do sistema. O arquivo de configuração deve ter pelo menos duas linhas (primeira linha, parâmetros do sistema, segunda linha parâmetros de uma tarefa), sendo que não há limite para o número de tarefas a serem executadas no sistema.

3.3.2. As strings contidas no arquivo de configurações devem ser tratadas de forma igual para o caso possuírem letras maiúsculas, minúsculas, ou a mistura de ambos. Exemplo: “PRIOP”, “priop” ou “PrioP” deve indicar a mesma configuração: escalonador preemptivo por prioridades.

3.3.3. A lista de eventos de cada tarefa vai ser tratada no projeto B. No entanto, deve-se atentar para o fato da possibilidade de ocorrência de inúmeros eventos ao longo da execução de uma tarefa.

- 3.4. O simulador deve permitir que o usuário modifique o estado atual de qualquer tarefa do sistema em qualquer passo/estágio da simulação.
4. Os seguintes algoritmos de escalonamento devem ser implementados: SRTF (string “SRTF” no arquivo de configurações), Prioridade preemptivo (string “PRIOP” no arquivo de configurações).
- 4.1. O usuário deve indicar, antes da simulação, qual é o algoritmo de escalonamento a ser usado;
 - 4.2. O mecanismo de implementação do algoritmo de escalonamento deve ser flexível/configurável/parametrizável de modo que novos algoritmos possam ser facilmente incluídos no simulador sem a necessidade de modificar o código da simulação. Idealmente, o escalonador é apenas uma função que retorna qual é a próxima tarefa a ser executada, que pode, inclusive, estar contida em uma biblioteca dinâmica fora do arquivo binário do executável do simulador;
 - 4.3. Em caso de empates na escolha da próxima tarefa (para todos os algoritmos), o desempate deve considerar os seguintes critérios, nessa ordem: (1) a tarefa selecionada é a que está executando imediatamente antes do escalonamento ser acionado (para evitar uma troca de contexto para a própria tarefa); (2) o instante de ingresso da tarefa: quem chegou antes é escolhida; (3) duração da tarefa: duração menor deve se escolhida; (4) sorteio (quando isso acontecer, um elemento gráfico deve aparecer no gráfico de gantt).
 - 4.4. Para o algoritmo de Prioridades Preemptivo, o primeiro critério de desempate é valor maior de prioridade estática indicando que a tarefa é mais prioritária. Se ainda houver empate, os critérios do item 4.3 devem considerados.
5. A interface de interação do usuário com o simulador deve ser intuitiva, fácil de entender e usar.

PROJETO B – Refinamento da Simulação do Sistema Operacional Multitarefa

Requisitos:

1. Ampliar os escalonadores suportados na simulação, incluindo a implementação do escalonador preemptivo por prioridades com envelhecimento.
 - 1.1. O parâmetro alpha deve ser configurável através do arquivo de configuração. A primeira linha do arquivo de configuração deverá ser no formato: `PRIOPEnv;5;2;1` onde o primeiro campo indica o nome do algoritmo, o segundo campo indica o *quantum* para as tarefas, e o terceiro campo indica a quantidade de processadores, e o quarto indica o valor do alpha usado para fazer o envelhecimento.
 - 1.2. Valores
 - 1.3. Em caso de empates na escolha da próxima tarefa, o desempate deve considerar os seguintes critérios, nessa ordem: (1) tarefa com maior prioridade estática; (2) a tarefa selecionada é a que está executando imediatamente antes do escalonamento ser acionado (para evitar uma troca de contexto para a própria tarefa); (2) o instante de ingresso da tarefa: quem chegou antes é escolhida; (3) duração da tarefa: duração menor deve se escolhida; (4) sorteio (quando isso acontecer, um elemento gráfico deve aparecer no gráfico de gantt).
2. Implementar as ações de solicitação e liberação de um *mutex*.
 - 2.1. As ações são descritas no arquivo de configuração para cada tarefa de descrita (linha do arquivo), após a especificação da prioridade das tarefas;
 - 2.2. A ação para indicar a solicitação do *mutex* é descrita no formato: `MLxx:00`, onde xx indica o numero do *mutex* e 00 indica o instante de tempo em que a ação acontece;
 - 2.3. A ação para indicar a liberar o *mutex* é descrita no formato: `MUxx:00`, onde xx indica o numero do *mutex* e 00 indica o instante de tempo em que a ação acontece
 - 2.4. Todos os instantes de tempo indicados nas ações são sempre relativos ao início da tarefa.
 - 2.5. Se houver mais de uma ação a ser executada no mesmo instante de tempo, elas devem ser executadas na ordem que foram especificadas no arquivo de configuração.
 - 2.6. Se uma ou mais ações acontecerem no instante zero, ou no instante em que uma tarefa (re)inicia a sua execução (troca de contexto), primeiro deve-se (re)iniciar a execução da tarefa e, logo em seguida, a(s) ação(ões) devem ser executadas considerando as restrições descritas no item 2.5.

- 2.7. As ações de solicitar e liberar o *mutex* devem ser tratados pelo sistema (durante a simulação) conforme o conteúdo estudado na disciplina.
 - 2.8. A execução das ações de mutex devem ser mostradas de forma clara na forma de um elemento gráfico no gráfico de gantt.
 - 2.9. O tempo que uma tarefa permanece suspensa deve ser mostrado em uma cor ou padrão de preenchimento diferente no gráfico de gantt (por exemplo, padrão de preenchimento quadriculado ao invés de sólido). Deve haver diferenciação gráfica entre suspensão por E/S e suspensão por mutex não disponível.
3. Implementar a ocorrência de operações de entrada e saída (E/S).
- 3.1. As ações são descritas no arquivo de configuração para cada tarefa de descrita (linha do arquivo), após a especificação da prioridade das tarefas;
 - 3.2. Uma operação de E/S descrita no formato: IO:xx-yy, onde xx indica o instante de tempo em que a operação acontece e yy indica o tempo de duração da operação, ou seja, quanto tempo a operação precisa para terminar de executar;
 - 3.3. Todos os instantes de tempo indicados na ocorrência da operação de E/S são sempre relativos ao início da tarefa.
 - 3.4. Todas as durações das operações de E/S são sempre relativos ao instante de tempo da ocorrência da operação. A duração mínima de qualquer ação de E/S é de uma unidade de tempo. Quando uma ação de E/S termina, ela deve gerar uma IRQ no instante de tempo imediatamente depois do instante de término, informando o Sistema Operacional que a operação de E/S terminou.
 - 3.5. Se houver mais de uma ação a ser executada no mesmo instante de tempo, elas devem ser executadas na ordem que foram especificadas no arquivo de configuração.
 - 3.6. Se uma ou mais ações acontecerem no instante zero, ou no instante em que uma tarefa (re)inicia a sua execução, primeiro deve-se (re)iniciar a execução da tarefa e, logo em seguida, a(s) ação(ões) devem ser executadas.
 - 3.7. As operações de E/S devem ser tratadas pelo sistema (durante a simulação) conforme o conteúdo estudado na disciplina.
 - 5.1. A execução das ações de E/S devem ser mostradas de forma clara na forma de um elemento gráfico no gráfico de gantt.
 - 3.8. O tempo que uma tarefa permanece suspensa deve ser mostrado em uma cor ou padrão de preenchimento diferente no gráfico de gantt (por exemplo, padrão de preenchimento quadriculado ao invés de sólido). Deve haver diferenciação gráfica entre suspensão por E/S e suspensão por mutex não disponível.